



ULPGC

**Universidad de
Las Palmas de
Gran Canaria**

eii

**ESCUELA DE
INGENIERÍA INFORMÁTICA**

Trabajo de Fin de Grado

Diseño y desarrollo de un sistema empotrado basado en bus CAN con acceso inalámbrico para la monitorización de un vehículo automóvil

TITULACIÓN: Grado en Ingeniería Informática

AUTOR: Gorka Eymard Santana Cabrera

TUTORIZADO POR:

Antonio Carlos Domínguez Brito, Jorge Cabrera Gámez

Fecha [03/2026]

SOLICITUD DE DEFENSA DE TRABAJO DE FIN DE TÍTULO

D./D^a Gorka Eymard Santana Cabrera, autor del trabajo Diseño y desarrollo de un sistema empujado basado en bus CAN con acceso inalámbrico para la monitorización de un vehículo automóvil, correspondiente a la titulación Grado de Ingeniería Informática.

SOLICITA

que se inicie el procedimiento de defensa del mismo, para lo que se adjunta la documentación requerida, haciendo constar que

☒ se autoriza / ☐ no se autoriza la grabación en audio de la exposición y turno de preguntas.

Asimismo, con respecto al registro de la propiedad intelectual/industrial del TFT, declara que:

☐ Se ha iniciado o hay intención de iniciarlo (defensa no pública).

☐ No está previsto.

Y para que así conste firma la presente. (fecha en firma electrónica)

El/La estudiante

Fdo. _____

A rellenar y firmar **obligatoriamente** por el/la/los/las tutores

En relación con la presente solicitud, se informa: (firmar donde corresponda)

Positivamente (en caso de detección de copia, esta firma quedará invalidada)

Fdo. _____

Negativamente (justificación en TFT05)

Fdo. _____

DIRECTOR DE LA ESCUELA DE INGENIERÍA INFORMÁTICA

Agradecimientos

La realización de este Trabajo de Fin de Título no habría sido posible sin el apoyo y la colaboración de diversas personas e instituciones a las que deseo expresar mi más sincero agradecimiento.

En primer lugar, quiero agradecer a los profesores Antonio C. Domínguez Brito y Jorge Cabrera Gámez, por su dedicación, orientación y apoyo durante el desarrollo de este trabajo. Su experiencia y los consejos recibidos han sido fundamentales para encauzar el desarrollo de este proyecto en la dirección correcta.

Asimismo, agradezco al profesor Jorge Cabrera Gámez, tutor colaborador, su implicación y el valioso apoyo técnico aportado durante las distintas fases del trabajo. Sus indicaciones han contribuido de manera significativa a la calidad del resultado final.

De igual modo, deseo expresar mi gratitud a la División de Robótica y Oceanografía Computacional del Instituto Universitario SIANI de la Universidad de Las Palmas de Gran Canaria, por poner a disposición de este proyecto los medios materiales y el equipamiento necesarios para llevar a cabo el desarrollo experimental y las pruebas del sistema.

Por último, agradezco a todas las personas que, de una forma u otra, han brindado su apoyo durante este período, haciendo posible la culminación de este trabajo.

Resumen

La estructura de este documento es orientativa. Las **partes obligatorias (motivación y objetivos, competencias, aportaciones, alineamiento ODS, desarrollo, conclusiones y bibliografía)** deben estar recogidas, aunque pueden reordenarse o redistribuirse en base a las características del trabajo.

EL presente Trabajo de Fin de Título(TFT) desarrolla un sistema empotrado basado en bus CAN que trabaja en la capa de aplicación de OBDII permitiendo la realización de diagnóstico de vehículos. El sistema ha sido diseñado y validado tomando como referencia la arquitectura de comunicaciones de los vehículos del grupo VAG (Volkswagen Audi Group), cuya topología de *Gateway* central es representativa de los vehículos modernos de la plataforma PQ25.

Abstract

Ahora en inglés: This Final Degree Project (TFT) develops an embedded system based on the CAN bus that operates at the OBD-II application layer, enabling vehicle diagnostics. The system has been designed and validated against the communication architecture of Volkswagen Audi Group (VAG) vehicles, whose central *Gateway* topology is representative of modern PQ25 platform vehicles.

Índice general

Listings	IV
1. Introducción	1
1.1. Contexto y motivación	1
1.2. Descripción del sistema desarrollado	2
1.3. Estructura del documento	2
2. Estado actual y objetivos	4
2.1. Motivación y antecedentes	4
2.2. Objetivos	5
2.2.1. Objetivo general	5
2.2.2. Objetivos específicos	5
2.2.3. Alcance y limitaciones	6

Listings

Índice de algoritmos

Índice de figuras

Índice de cuadros

2.1. Trazabilidad entre funciones definidas en la propuesta TFT01 y objetivos específicos del trabajo.	5
--	---

Capítulo 1

Introducción

1.1. Contexto y motivación

El sector automovilístico ha seguido una tendencia clara hacia la sensorización y comunicación de datos en tiempo real, impulsada por la necesidad de monitorizar el estado del vehículo y anticipar posibles fallos antes de que se conviertan en problemas graves. Esta tendencia ha consistido en la incorporación de sensores cada vez más sofisticados, tales como sensores de presión, temperatura, posición. Estos generan una cantidad voluminosa de datos que deben ser procesados y transmitidos entre las distintas Unidades de Control Electrónico (ECU, del inglés *Electronic Control Unit*) que orquestan el funcionamiento del vehículo además estos datos han de ser tratados según su prioridad y criticidad, lo que ha llevado a la adopción de protocolos de comunicación robustos y eficientes, .

Para lograr una comunicación fiable, en tiempo real y con gestión de prioridades los fabricantes comenzaron a implementar el bus CAN (*Controller Area Network*), un protocolo de comunicaciones serie creado por Robert Bosch GmbH en 1983 y estandarizado bajo la norma ISO 11898. Esencialmente el bus CAN consiste en un protocolo que permite la comunicación de múltiples microcontroladores compartiendo un mismo medio y donde se resuelven los conflictos de transmisión mediante un mecanismo de prioridades. Un ejemplo en la práctica podría ser la comunicación entre la ECU de seguridad, como el sistema antibloqueo de frenos (ABS), y la unidad de control del motor: si el ABS detecta que una rueda resbala, envía un mensaje prioritario por el bus CAN para que el motor reduzca su potencia inmediatamente.

Por otra parte en 1996 se introdujo el estándar OBD-II (*On-Board Diagnostics, segunda generación*), que define un conjunto de protocolos de comunicación junto con una batería de códigos de error comunes (DTCs - *Diagnostic Trouble Code*). Así mismo permite la monitorización y evaluación del estado de los microcontroladores integrados en el vehículo.

1.2. Descripción del sistema desarrollado

Este proyecto se ha centrado en el diseño y desarrollo de un sistema de diagnóstico de vehículos que usan el protocolo OBD-II sobre bus CAN y que ofrece acceso inalámbrico usando Bluetooth Low Energy (BLE). Durante el desarrollo se han implementado tres componentes principales:

El componente principal es la **herramienta de diagnóstico**, creada sobre una Raspberry Pi conectada al bus CAN usando el CAN HAT rs485 que cuenta con controlador mcp215. Esta herramienta implementa los siguientes protocolos:

- ✓ **ISO-TP (ISO 15765-2)** como capa de transporte sobre CAN.
- ✓ **OBD-II (SAE J1979)** como capa de aplicación.

La herramienta decodifica los parámetros recibidos, los almacena en una base de datos SQLite y los transmite de forma inalámbrica usando BLE y un servidor GATT que implementa el servicio Nordic UART Service (NUS).

El componente que ha permitido realizar pruebas es un **simulador de ECU** implementado sobre una plataforma Arduino MKRWAN 1310 con un shield CAN.

El tercer componente es una **aplicación móvil nativa** desarrollada con React Native y Expo, compatible con Android e iOS, que se conecta a la Raspberry Pi vía Bluetooth Low Energy. La aplicación ofrece al usuario una interfaz para la monitorización en tiempo real de los parámetros del vehículo, la consulta y gestión de códigos de avería, asimismo también presenta una pantalla para depuración que permite al usuario consultar el funcionamiento de la comunicación.

El conjunto del sistema da lugar a un sistema de diagnóstico OBD-II portable, inalámbrica y de bajo coste, validado sobre un banco de pruebas basado en Arduino antes de su uso sobre hardware vehicular real y permitiendo realizar experimentos sin acceso a un vehículo real.

1.3. Estructura del documento

El presente documento se organiza en siete capítulos cuyo contenido se resume a continuación.

El **Capítulo 2** recoge la motivación por la cual se ha decidido realizar el trabajo y la definición de los objetivos que se pretenden alcanzar.

El **Capítulo 3** describe las principales aportaciones del trabajo, las competencias específicas del Grado de Ingeniería Informática desarrolladas durante su realización y la relación del proyecto con los Objetivos de Desarrollo Sostenible de la ONU.

El **Capítulo 4** presenta la metodología de desarrollo seguida, basada en prototipos incrementales, y detalla las fases y tareas en las que se estructuró el trabajo, incluyendo una

valoración de las desviaciones respecto a la planificación inicial.

El **Capítulo 5** constituye el núcleo técnico de la memoria. Describe en detalle los protocolos de comunicación empleados (CAN, ISO-TP, OBD-II y BLE GATT), el diseño e implementación de cada uno de los tres componentes del sistema, y las pruebas de integración realizadas.

El **Capítulo 6** presenta los resultados obtenidos, incluyendo métricas de rendimiento y una evaluación del grado de consecución de los objetivos planteados.

El **Capítulo 7** recoge las conclusiones del trabajo, las líneas de trabajo futuro identificadas y una reflexión sobre el uso de herramientas de inteligencia artificial durante el desarrollo.

Finalmente, el documento incluye la bibliografía de referencia y cuatro anexos con fragmentos representativos del código fuente desarrollado y el esquema de la base de datos.

Capítulo 2

Estado actual y objetivos

2.1. Motivación y antecedentes

En el actual plan de estudios del Grado de Ingeniería Informática en la asignatura de Internet de las cosas se estudia el ecosistema de los sistemas empotrados y sus múltiples protocolos de comunicación. Dentro de la amplia gama de protocolos de comunicación el estudio del bus CAN y se comentó su uso en el sector automotriz, esto dió lugar a la motivación por la cual se ha desarrollado este Trabajo de Fin de Título en el que se estudia el uso del bus CAN y el protocolo OBD-II y demás protocolos que orquestan la comunicación de información en un vehículo.

Actualmente los automóviles cuentan con un sistema de numerosos microcontroladores que generan una cantidad muy voluminosa de datos de telemetría y además cuentan con el protocolo de diagnóstico abordo (OBD-II). Estos son accesibles a través del puerto OBD que se encuentra normalmente en la parte inferior del volante en el asiento del piloto. Este puerto permite que no solo los talleres y personas especializadas en el mundo de la mecánica y electromecánica, sino que cualquier persona con un "escáner" pueda ver el estado de su coche o pueda diagnosticar que fallo tiene o anticipar posibles fallos.

Actualmente en el mercado han aparecido numerosas soluciones para poner de forma accesible el acceso al diagnóstico OBD-II. La mayor parte de estas soluciones usan el adaptador ELM327, montando un circuito integrado que actúa como puente entre el bus CAN del vehículo y una interfaz accesible vía Bluetooth o Wi-Fi. Por otra parte existen numerosas aplicaciones móviles compatibles con estos adaptadores tales como Torque Pro u OBD Fusion, no obstante estas soluciones no suelen permitir el registro estructurado de datos para análisis y la mayor parte de estas soluciones introducen latencias significativas dando lugar a una experiencia de usuario pobre.

2.2. Objetivos

A partir del contexto descrito, se definen los siguientes objetivos para este proyecto.

2.2.1. Objetivo general

Diseñar y desarrollar un sistema empotrado de diagnóstico del vehículo basado en el protocolo OBD-II sobre bus CAN, con acceso inalámbrico vía Bluetooth Low Energy, de bajo coste económico y validado mediante un banco de pruebas basado en Arduino.

2.2.2. Objetivos específicos

La propuesta de este trabajo (TFT01) define tres funciones de usuario de alto nivel que el sistema debe satisfacer. La tabla 2.1 recoge estas funciones junto con los objetivos específicos de la memoria que las implementan.

Cuadro 2.1: Trazabilidad entre funciones definidas en la propuesta TFT01 y objetivos específicos del trabajo.

Función TFT01	Descripción	Obj.
Monitorización en tiempo real	Visualización numérica de parámetros de sensores disponibles en el vehículo (RPM, temperaturas, presiones, voltajes, etc.)	O1, O2, O4, O5, O6
Consulta y gestión de errores	Lectura y borrado de códigos de error (DTCs) almacenados en la memoria de las ECUs	O1, O2, O3, O4, O6
Identificación	Consulta de datos del vehículo (VIN) e información de los distintos módulos integrados	O1, O2, O4, O5

Los seis objetivos específicos que concretan estas funciones son:

- O1. Implementar la pila de protocolos OBD-II completa:** desarrollar sobre Raspberry Pi una implementación funcional de ISO-TP (ISO 15765-2) como capa de transporte y OBD-II (SAE J1979) como capa de aplicación, cubriendo los modos de servicio 0x01 (datos en vivo), 0x03 (lectura de DTCs), 0x04 (borrado de DTCs) y 0x09 (información del vehículo, incluyendo VIN).
- O2. Desarrollar un simulador de ECU:** implementar sobre Arduino MKR con shield CAN un simulador de ECU que responda a peticiones OBD-II estándar con datos de telemetría realistas, con el objetivo de disponer de un banco de pruebas reproducible y controlado que no dependa de la disponibilidad de un vehículo real.

- O3. Diseñar mecanismos de simulación de condiciones adversas:** incorporar en el simulador Arduino la generación de ruido CAN (pérdida de tramas, latencia variable, respuestas negativas esporádicas) y tráfico de fondo aleatorio, para validar la robustez de la herramienta de diagnóstico.
- O4. Proporcionar acceso inalámbrico mediante BLE:** implementar un servidor BLE GATT sobre la Raspberry Pi que exponga la funcionalidad de diagnóstico a través del servicio Nordic UART Service (NUS), eliminando la necesidad de conexión física entre el dispositivo de diagnóstico y el usuario.
- O5. Desarrollar una aplicación móvil multiplataforma:** implementar una aplicación para Android e iOS con React Native que permita al usuario conectarse al sistema de diagnóstico vía BLE, monitorizar parámetros del vehículo en tiempo real, gestionar los códigos de avería y consultar el historial de sesiones almacenadas.
- O6. Registrar y almacenar los datos de diagnóstico:** implementar un sistema de logging estructurado en base de datos SQLite que almacene las sesiones, las muestras de telemetría y los comandos ejecutados, para su consulta y análisis posterior.

2.2.3. Alcance y limitaciones

El sistema desarrollado se centra en la implementación de los modos OBD-II estándar definidos por SAE J1979, sin abordar los servicios de diagnóstico propietarios de cada fabricante, que requieren herramientas específicas y, en la mayor parte de los casos, acuerdos de licencia con el fabricante del vehículo. Asimismo, la validación experimental del sistema se realiza sobre el banco de pruebas desarrollado en Arduino, dado que las restricciones de acceso al hardware del vehículo real durante el período de desarrollo del trabajo no permitieron completar las pruebas sobre el vehículo objetivo. Las implicaciones de esta limitación y las líneas de trabajo futuro que abre se discuten en el Capítulo 7.

Bibliografía

- [1] Robert Bosch GmbH, *CAN Specification Version 2.0*, Robert Bosch GmbH, Stuttgart, 1991.
- [2] International Organization for Standardization, *ISO 11898-1:2015 – Road vehicles – Controller area network (CAN) – Part 1: Data link layer and physical signalling*, ISO, Ginebra, 2015.
- [3] International Organization for Standardization, *ISO 15765-2:2016 – Road vehicles – Diagnostic communication over Controller Area Network (DoCAN) – Part 2: Transport protocol and network layer services*, ISO, Ginebra, 2016.
- [4] Society of Automotive Engineers, *SAE J1979:2012 – E/E Diagnostic Test Modes*, SAE International, Warrendale, 2012.
- [5] International Organization for Standardization, *ISO 14229-1:2020 – Road vehicles – Unified diagnostic services (UDS) – Part 1: Application layer*, ISO, Ginebra, 2020.
- [6] S. Ménager *et al.*, *python-can-isotp Documentation*, 2023. Disponible en: <https://can-isotp.readthedocs.io/>
- [7] python-can contributors, *python-can Documentation*, 2023. Disponible en: <https://python-can.readthedocs.io/>